



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение  
высшего образования

«МИРЭА – Российский технологический университет»

**РТУ МИРЭА**

---

Институт Информационных технологий

Кафедра математического обеспечения и стандартизации  
информационных технологий

## **Отчет по практической работе №1**

по дисциплине «Тестирование и верификация программного обеспечения»

**Выполнил:**

Студент группы ИКБО-32-23

Беляев А.Е.

**Проверил:**

Ильичев Г.П.

**Отчет представлен**

«\_\_\_»\_\_\_\_\_2025г.

Москва 2025

## СОДЕРЖАНИЕ

|                                           |    |
|-------------------------------------------|----|
| 1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПРИЛОЖЕНИЕ..... | 3  |
| 2. ЗАДАЧА ДЛЯ TDD.....                    | 4  |
| 3. ЗАДАЧА ДЛЯ ATDD.....                   | 9  |
| .....                                     | 13 |
| 4. ЗАДАЧА ДЛЯ BDD.....                    | 14 |
| 5. ЗАДАЧА ДЛЯ SDD.....                    | 18 |
| 6. ОБЩИЕ ВЫВОДЫ ПО МЕТОДАМ.....           | 23 |

# 1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА ПРИЛОЖЕНИЕ

**Название:** Навигатор с функциями поиска местоположения, построения и отображения маршрута.

**Цель:** Разработать модуль навигации, обеспечивающий:

поиск точки по адресу;  
построение оптимального маршрута между двумя точками;  
корректное отображение маршрута на карте.

**Функциональные требования:**

1. **Поиск местоположения** — преобразование адреса в координаты (узел графа).
2. **Построение маршрута** — нахождение кратчайшего пути по расстоянию между двумя узлами.
3. **Отображение маршрута** — возврат списка узлов и общей длины пути для визуализации.

**Нефункциональные требования:**

- Алгоритм использует взвешенный граф.
- Обработка ошибок: отсутствие пути, некорректные адреса.
- Покрытие тестами  $\geq 90\%$ .
- Допустимо использование networkx.

**Технологии:** Python 3.14, networkx, unittest.pytest,hypothesis,behave

## 2. ЗАДАЧА ДЛЯ TDD

**Цель:** Гарантировать корректность функции `build_route(graph, start, end)`, возвращающей:

- список узлов оптимального пути;
- общую длину пути;
- исключение `ValueError` при отсутствии пути.

### Шаг 1. Создание тестов (Red)

Напишем Юнит-тесты для TDD. На данном этапе Программа будет выводить исключительно одни ошибки, что закономерно при отсутствии какой-либо логики приложения. Юнит-тесты я запускал с флагом `-v` для более «человеческого» отображения результатов. Напишем Юнит-тесты(рис.1):

```
'''
import unittest
from navigator import build_route

class TestRouteBuilding(unittest.TestCase):

    def test_direct_connection(self):
        graph = {'A': {'B': 10}}
        path, dist = build_route(graph, 'A', 'B')
        self.assertEqual(path, ['A', 'B'])
        self.assertEqual(dist, 10)

    def test_multiple_hops_optimal_path(self):
        graph = {
            'A': {'B': 5, 'C': 15},
            'B': {'C': 3},
            'C': {}
        }
        path, dist = build_route(graph, 'A', 'C')
        self.assertEqual(path, ['A', 'B', 'C'])
        self.assertEqual(dist, 8)

    def test_no_path_exists(self):
        graph = {'A': {'B': 5}, 'B': {}}
        with self.assertRaises(ValueError):
            build_route(graph, 'A', 'C')

    def test_same_start_and_end(self):
        graph = {'A': {}}
        path, dist = build_route(graph, 'A', 'A')
        self.assertEqual(path, ['A'])
        self.assertEqual(dist, 0)

if __name__ == '__main__':
    unittest.main(verbosity=2)
'''
```

```
test_navigator_tdd.py ×
1  import unittest
2  from navigator import build_route
3
4
5  class TestRouteBuilding(unittest.TestCase):
6
7      def test_direct_connection(self):
8          graph = {'A': {'B': 10}}
9          path, dist = build_route(graph, 'A', 'B')
10         self.assertEqual(path, ['A', 'B'])
11         self.assertEqual(dist, 10)
12
13     def test_multiple_hops_optimal_path(self):
14         graph = {
15             'A': {'B': 5, 'C': 15},
16             'B': {'C': 3},
17             'C': {}
18         }
19         path, dist = build_route(graph, 'A', 'C')
20         self.assertEqual(path, ['A', 'B', 'C'])
21         self.assertEqual(dist, 8)
22
23     def test_no_path_exists(self):
24         graph = {'A': {'B': 5}, 'B': {}}
25         with self.assertRaises(ValueError):
26             build_route(graph, 'A', 'C')
27
28     def test_same_start_and_end(self):
29         graph = {'A': {}}
30         path, dist = build_route(graph, 'A', 'A')
31         self.assertEqual(path, ['A'])
32         self.assertEqual(dist, 0)
33
34
35 if __name__ == '__main__':
36     unittest.main(verbosity=2)
```

Рисунок 1 — Написание Юнит-тестов

И конечно же составим само приложение (я бы мог его и не создавать, результат особо не поменялся, но все ошибки возникали просто из-за отсутствия самого файла, но это совсем неинтересно, поэтому я положу в единственную функцию вывод `print(1)`(рис.2).

...

```
import networkx as nx

def build_route(graph_dict: Dict[str, Dict[str, float]], start: str, end: str) -> Tuple[List[str], float]:
    print(1)
```

...

```
import networkx as nx

def build_route(graph_dict: Dict[str, Dict[str, float]], start: str, end: str) -> Tuple[List[str], float]:
    print(1)
```

Рисунок 2-Мусорный код в приложении

На выводе получим такую картину(рис.3):

```
(.venv) PS C:\Users\aleks\PycharmProjects\PythonProject> python test_navigator_tdd.py -v
test_direct_connection (__main__.TestRouteBuilding.test_direct_connection) ... 1
ERROR
test_multiple_hops_optimal_path (__main__.TestRouteBuilding.test_multiple_hops_optimal_path) ... 1
ERROR
test_no_path_exists (__main__.TestRouteBuilding.test_no_path_exists) ... 1
FAIL
test_same_start_and_end (__main__.TestRouteBuilding.test_same_start_and_end) ... 1
ERROR
```

Рисунок 3 — все тесты провалились

## Шаг 2. Минимальная реализация (Green)

Теперь создадим минимальную реализацию для TDD(рис.4):

```
'''
```

```
# navigator.py
import networkx as nx

def build_route(graph_dict: dict, start: str, end: str):
    if start == end:
        return [start], 0

    G = nx.Graph()
    for node, edges in graph_dict.items():
        for neighbor, weight in edges.items():
            G.add_edge(node, neighbor, weight=weight)

    try:
        path = nx.shortest_path(G, source=start, target=end, weight='weight')
        distance = nx.shortest_path_length(G, source=start, target=end, weight='weight')
        return path, distance
    except nx.NetworkXNoPath:
        raise ValueError("No route exists between start and end")

'''
```

```
test_navigator_tdd.py  navigator.py x
1  import networkx as nx
2
3
4  def build_route(graph_dict: dict, start: str, end: str): 5 usages
5      if start == end:
6          return [start], 0
7
8      G = nx.Graph()
9      for node, edges in graph_dict.items():
10         for neighbor, weight in edges.items():
11             G.add_edge(node, neighbor, weight=weight)
12
13     try:
14         path = nx.shortest_path(G, source=start, target=end, weight='weight')
15         distance = nx.shortest_path_length(G, source=start, target=end, weight='weight')
16         return path, distance
17     except nx.NetworkXNoPath:
18         raise ValueError("No route exists between start and end")
```

Рисунок 4 — Green реализация

Вывод такой реализации показывает 1 ошибку из-за ошибки, связанной с узлом C, необходимым для работы networkx. Это подходит для прохода этапа Green, не требующего полной работы продукта(рис.5).

```
(.venv) PS C:\Users\aleks\PycharmProjects\PythonProject> python test_navigator_tdd.py -v
test_direct_connection (__main__.TestRouteBuilding.test_direct_connection) ... ok
test_multiple_hops_optimal_path (__main__.TestRouteBuilding.test_multiple_hops_optimal_path) ... ok
test_no_path_exists (__main__.TestRouteBuilding.test_no_path_exists) ... ERROR
test_same_start_and_end (__main__.TestRouteBuilding.test_same_start_and_end) ... ok
```

Рисунок 5 — результаты для Green

Как видно из вывода, не прошла только функция test-no-path-exists, пофиксим это на этапе Refactoring.

### Шаг 3. Refactoring

Теперь, добавим обработку ошибки для несуществующей точки C, тем самым выполнив все юнит-тесты(рис.6):

...

```
# navigator.py
import networkx as nx
from typing import Dict, List, Tuple

def build_route(graph_dict: Dict[str, Dict[str, float]], start: str, end: str) -> Tuple[List[str], float]:
    if not graph_dict:
        raise ValueError("Graph is empty")

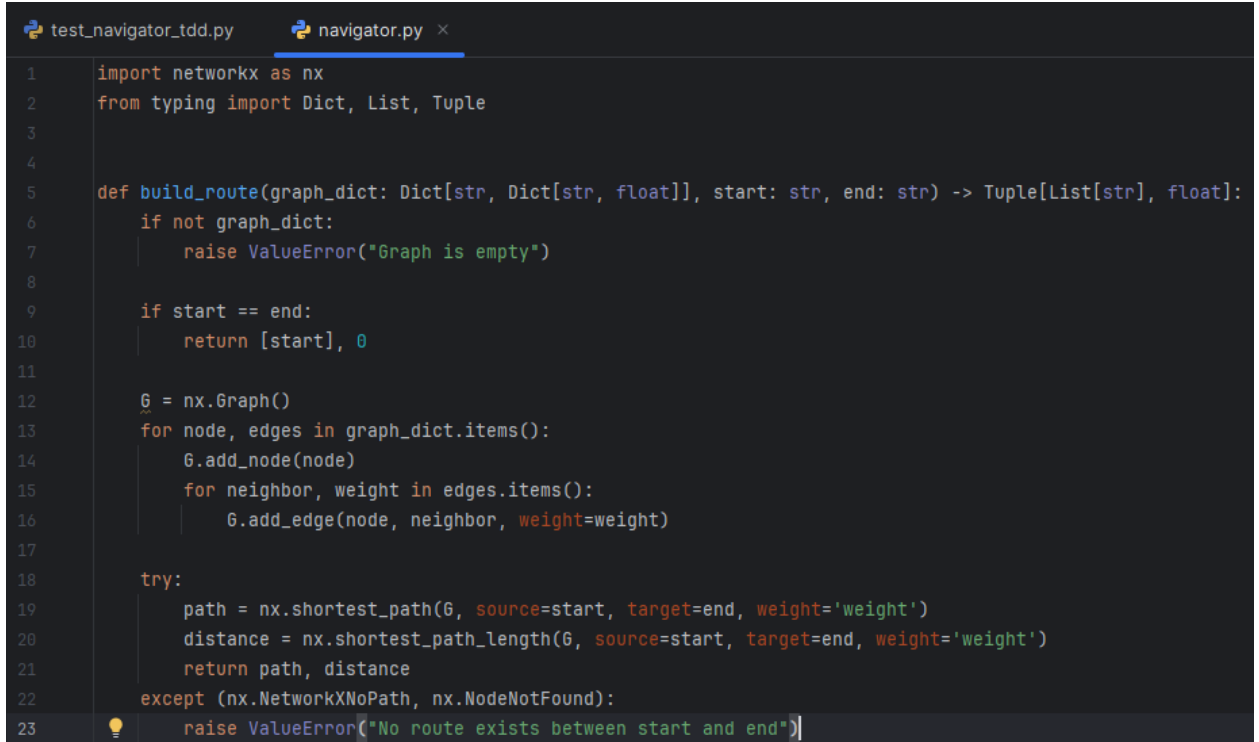
    if start == end:
        return [start], 0

    G = nx.Graph()
    for node, edges in graph_dict.items():
        G.add_node(node)
        for neighbor, weight in edges.items():
            G.add_edge(node, neighbor, weight=weight)
```

```

try:
    path = nx.shortest_path(G, source=start, target=end, weight='weight')
    distance = nx.shortest_path_length(G, source=start, target=end, weight='weight')
    return path, distance
except (nx.NetworkXNoPath, nx.NodeNotFound):
    raise ValueError("No route exists between start and end")
'''

```



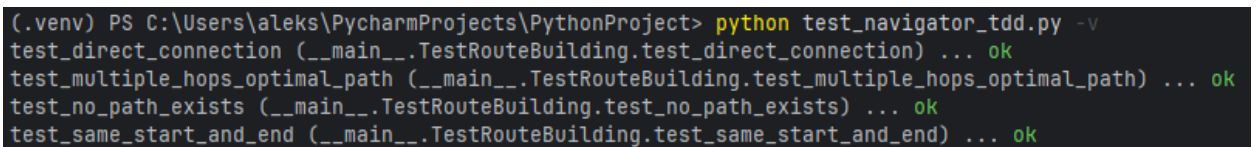
```

test_navigator_tdd.py  navigator.py x
1  import networkx as nx
2  from typing import Dict, List, Tuple
3
4
5  def build_route(graph_dict: Dict[str, Dict[str, float]], start: str, end: str) -> Tuple[List[str], float]:
6      if not graph_dict:
7          raise ValueError("Graph is empty")
8
9      if start == end:
10         return [start], 0
11
12     G = nx.Graph()
13     for node, edges in graph_dict.items():
14         G.add_node(node)
15         for neighbor, weight in edges.items():
16             G.add_edge(node, neighbor, weight=weight)
17
18     try:
19         path = nx.shortest_path(G, source=start, target=end, weight='weight')
20         distance = nx.shortest_path_length(G, source=start, target=end, weight='weight')
21         return path, distance
22     except (nx.NetworkXNoPath, nx.NodeNotFound):
23         raise ValueError("No route exists between start and end")

```

Рисунок 6 — рефакторинг

Вывод такой программы покажет 100% проходимость(рис 7)



```

(.venv) PS C:\Users\aleks\PycharmProjects\PythonProject> python test_navigator_tdd.py -v
test_direct_connection (__main__.TestRouteBuilding.test_direct_connection) ... ok
test_multiple_hops_optimal_path (__main__.TestRouteBuilding.test_multiple_hops_optimal_path) ... ok
test_no_path_exists (__main__.TestRouteBuilding.test_no_path_exists) ... ok
test_same_start_and_end (__main__.TestRouteBuilding.test_same_start_and_end) ... ok

```

Рисунок 7 — вывод на рефакторинге

Итого, в рефакторинге:

1. Добавлено явное добавление узлов через `G.add_node(node)`
2. Объединены исключения: `NetworkXNoPath` и `NodeNotFound`
3. Добавлены type hints, docstring, проверка на пустой граф

## Заключение по TDD

TDD позволил обнаружить баг с отсутствующим узлом графа, гарантировать 100% выполняемость кода с обработкой ошибок и создать чёткую, документированную архитектуру.



### 3. ЗАДАЧА ДЛЯ ATDD

**Цель:** Гарантировать, что функция `display_route(path, distance)` возвращает корректную строку отображения, соответствующую требованиям заказчика (формат "Route: [path] , Distance: [dist] km").

#### Шаг 1. Обсуждение требований

Собраны сценарии с заказчиком:

- Успешное отображение простого маршрута.
- Отображение маршрута с несколькими точками.
- Обработка нулевого расстояния.
- Ошибка при отсутствии пути.

#### Шаг 2. Создание приёмочных тестов (Red)

Создадим приёмочные тесты в соответствии требованиям заказчика:

```

```
# test_navigator_atdd.py
import pytest
from navigator import build_route, display_route

@pytest.fixture
def graph():
    return {
        'A': {'B': 5, 'C': 15},
        'B': {'C': 3},
        'C': {}
    }

def test_display_simple_route(graph):
    path, dist = build_route(graph, 'A', 'B')
    result = display_route(path, dist)
    assert result == "Route: A -> B, Distance: 5 km"

def test_display_multi_hop_route(graph):
    path, dist = build_route(graph, 'A', 'C')
    result = display_route(path, dist)
    assert result == "Route: A -> B -> C, Distance: 8 km"

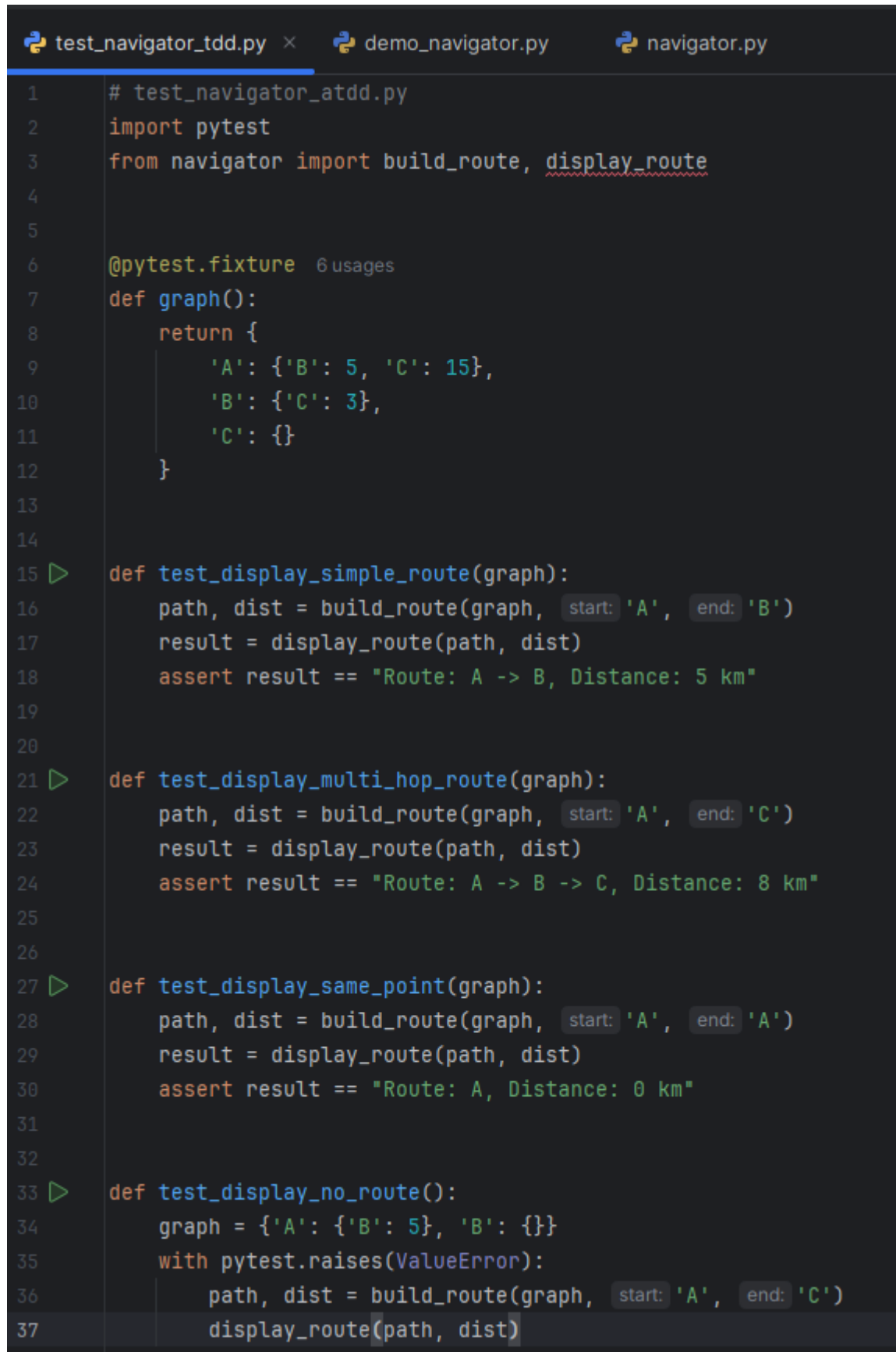
def test_display_same_point(graph):
    path, dist = build_route(graph, 'A', 'A')
    result = display_route(path, dist)
    assert result == "Route: A, Distance: 0 km"

def test_display_no_route():
    graph = {'A': {'B': 5, 'B': {}}}
    with pytest.raises(ValueError):
        path, dist = build_route(graph, 'A', 'C')
        display_route(path, dist)
```

```

На данном этапе сам продукт я оставил в таком состоянии, в каком он был на

конец рефакторинга TDD(рис.8-9)



```
test_navigator_tdd.py x demo_navigator.py navigator.py
1 # test_navigator_atdd.py
2 import pytest
3 from navigator import build_route, display_route
4
5
6 @pytest.fixture 6 usages
7 def graph():
8     return {
9         'A': {'B': 5, 'C': 15},
10        'B': {'C': 3},
11        'C': {}
12    }
13
14
15 def test_display_simple_route(graph):
16     path, dist = build_route(graph, start: 'A', end: 'B')
17     result = display_route(path, dist)
18     assert result == "Route: A -> B, Distance: 5 km"
19
20
21 def test_display_multi_hop_route(graph):
22     path, dist = build_route(graph, start: 'A', end: 'C')
23     result = display_route(path, dist)
24     assert result == "Route: A -> B -> C, Distance: 8 km"
25
26
27 def test_display_same_point(graph):
28     path, dist = build_route(graph, start: 'A', end: 'A')
29     result = display_route(path, dist)
30     assert result == "Route: A, Distance: 0 km"
31
32
33 def test_display_no_route():
34     graph = {'A': {'B': 5}, 'B': {}}
35     with pytest.raises(ValueError):
36         path, dist = build_route(graph, start: 'A', end: 'C')
37         display_route(path, dist)
```

Рисунок 8 — Написание юнит-тестов для ATDD

```
test_navigator_tdd.py  demo_navigator.py  navigator.py x
1  import networkx as nx
2  from typing import Dict, List, Tuple
3
4  def build_route(graph_dict: Dict[str, Dict[str, float]], start: str, end: str) -> Tuple[List[str], float]:
5      if not graph_dict:
6          raise ValueError("Graph is empty")
7
8      if start == end:
9          return [start], 0
10
11     G = nx.Graph()
12     for node, edges in graph_dict.items():
13         G.add_node(node)
14         for neighbor, weight in edges.items():
15             G.add_edge(node, neighbor, weight=weight)
16
17     try:
18         path = nx.shortest_path(G, source=start, target=end, weight='weight')
19         distance = nx.shortest_path_length(G, source=start, target=end, weight='weight')
20         return path, distance
21     except (nx.NetworkXNoPath, nx.NodeNotFound):
22         raise ValueError("No route exists between start and end")
```

Рисунок 9 — нынешний код приложения

И естественно, вывод будет полностью провален, из-за отсутствия нужных импортов(рис.10)

```
from navigator import build_route, display_route
E ImportError: cannot import name 'display_route' from 'navigator' (C:\Users\aleks\PycharmProjects\PythonProject\navigator.py)
collected 0 items / 1 error

!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!!!
===== 1 error in 0.31s =====
```

Рисунок 10 — вывод Red

### Шаг 3. Реализация функциональности (Green)

Обновим наш продукт, добавив весь необходимый функционал(рис.11)

```
test_navigator_tdd.py  demo_navigator.py  navigator.py x
1  import networkx as nx
2  from typing import Dict, List, Tuple
3
4
5  def display_route(path: List[str], distance: float) -> str: 10 usages
6      route_str = " -> ".join(path)
7      return f"Route: {route_str}, Distance: {distance} km"
8
9  def build_route(graph_dict: Dict[str, Dict[str, float]], start: str, end: str) -> Tuple[List[str], float]:
10
11      if not graph_dict:
12          raise ValueError("Graph is empty")
13
14      if start == end:
15          return [start], 0
16
17      G = nx.Graph()
18      for node, edges in graph_dict.items():
19          G.add_node(node)
20          for neighbor, weight in edges.items():
21              G.add_edge(node, neighbor, weight=weight)
22
23      try:
24          path = nx.shortest_path(G, source=start, target=end, weight='weight')
25          distance = nx.shortest_path_length(G, source=start, target=end, weight='weight')
26          return path, distance
27      except (nx.NetworkXNoPath, nx.NodeNotFound):
28          raise ValueError("No route exists between start and end")
```

Рисунок 11-добавление функционала

Вывод при таком коде покроет 100% тестов(рис.12)

```
===== test session starts =====
collecting ... collected 4 items

test_navigator_tdd.py::test_display_simple_route PASSED [ 25%]
test_navigator_tdd.py::test_display_multi_hop_route PASSED [ 50%]
test_navigator_tdd.py::test_display_same_point PASSED [ 75%]
test_navigator_tdd.py::test_display_no_route PASSED [100%]

===== 4 passed in 0.13s =====

Process finished with exit code 0
```

Рисунок 12 — результаты юнит-тестов

## Шаг 5. Рефакторинг

Доведём наш продукт до идеала, отрефакторив код. Просто обновлю функцию `display_route`(рис.13)

```

✓ def display_route(path: List[str], distance: float) -> str: 10 usages
✓     if not path:
        raise ValueError("Invalid path")
    route_str = " -> ".join(path)
    unit = "km" if distance > 0 else "km (no movement)"
    return f"Route: {route_str}, Distance: {distance} {unit}"

```

Рисунок 13 — рефакторинг функции

Вывод показал 100%(рис.14)

```

===== test session starts =====
collecting ... collected 4 items

test_navigator_tdd.py::test_display_simple_route PASSED [ 25%]
test_navigator_tdd.py::test_display_multi_hop_route PASSED [ 50%]
test_navigator_tdd.py::test_display_same_point PASSED [ 75%]
test_navigator_tdd.py::test_display_no_route PASSED [100%]

===== 4 passed in 0.13s =====

```

Рисунок 14 - вывод после рефакторинга

### Закключение по ATDD:

Этот метод позволил быстро и чётко построить юнит-тесты удовлетворяющие сразу условиям заказчика и бизнес-аналитиков, ускорив и упростив процесс разработки и тестирования.

## 4. ЗАДАЧА ДЛЯ BDD

**Цель:** Описать поведение системы в естественном языке, подтвердить корректность поиска, построения и отображения маршрута.

### Шаг 1. Формулирование сценариев на естественном языке

Сценарии в формате Gherkin (Given-When-Then):

- Успешный поиск маршрута.
- Поиск без маршрута.

### Шаг 2. Написание сценариев (Red)

Напишем автоматизированные сценарии по нашим требованиям. Для этого я буду использовать язык Gherkin, требующий наличия директории features, и поддиректории steps с логикой работы каждой фичи. Для начала создадим файл со сценариями в папке features(рис.15)

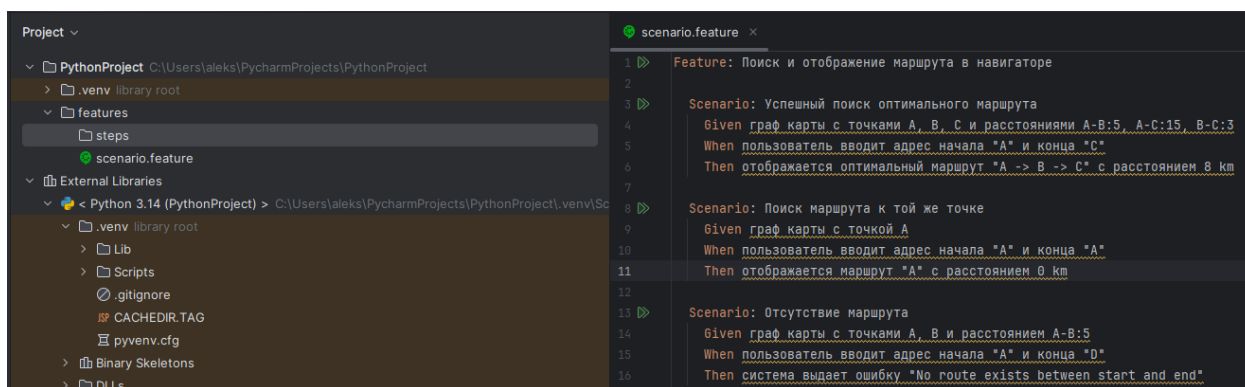


Рисунок 15 — создание сценариев

Запустив тест сценариев через behave в терминале мы получаем проваленные тесты(рис.16)

```
Errored scenarios:
features/scenario.feature:3 Успешный поиск оптимального маршрута
features/scenario.feature:8 Поиск маршрута к той же точке
features/scenario.feature:13 Отсутствие маршрута

0 features passed, 0 failed, 1 error, 0 skipped
0 scenarios passed, 0 failed, 3 error, 0 skipped
0 steps passed, 0 failed, 0 skipped, 9 undefined
Took 0min 0.000s
```

Рисунок 16 — вывод сценариев

### Шаг 3. Реализация поведения (Green)

Напишем логику приложения для сценариев, где опишем поведение программы, логику работы пока не трогаем(рис.17)

```
scenario.feature  navigator_steps.py  navigator.py
4  @given('граф карты с точками {points} и расстояниями {distances}')
5  def step_given_graph(context, points, distances):
6      context.graph = {}
7      # Парсинг: "A, B, C" -> ['A', 'B', 'C']
8      nodes = [n.strip() for n in points.split(',')]
9      for node in nodes:
10         context.graph[node] = {}
11      # Парсинг: "A-B:5, A-C:15, B-C:3"
12      for dist in distances.split(', '):
13         src_dest, weight = dist.split(':')
14         src, dest = src_dest.split('-')
15         context.graph[src][dest] = float(weight)
16         context.graph[dest][src] = float(weight) # Неориентированный граф
17
18  @given('граф карты с точкой {point}')
19  def step_given_single_point(context, point):
20      context.graph = {point: {}}
21
22  @when('пользователь вводит адрес начала "{start}" и конца "{end}"')
23  def step_when_search_route(context, start, end):
24      try:
25         context.path, context.dist = build_route(context.graph, start, end)
26         context.result = display_route(context.path, context.dist)
27     except ValueError as e:
28         context.error = str(e)
29
30  @then('отображается оптимальный маршрут "{expected_path}" с расстоянием {expected_dist} km')
31  def step_then_display_route(context, expected_path, expected_dist):
32      expected = f"Route: {expected_path}, Distance: {expected_dist} km"
33      assert context.result == expected
34
35  @then('отображается маршрут "{expected_path}" с расстоянием {expected_dist} km')
36  def step_then_display_same_point(context, expected_path, expected_dist):
37      expected = f"Route: {expected_path}, Distance: {expected_dist} km"
38      assert context.result == expected
39
40  @then('система выдает ошибку "{expected_error}"')
41  def step_then_error(context, expected_error):
42      assert context.error == expected_error
```

Рисунок 17 — сценарии

Посмотрим на вывод программы(рис.18)

```
Feature: Поиск и отображение маршрута в навигаторе # features/scenario.feature:1
  Then отображается оптимальный маршрут "A -> B -> C" с расстоянием 8 km # features/steps/navigator_steps.py:30 0.000s
  ASSERT FAILED:я оптимальный маршрут "A -> B -> C" с расстоянием 8 km # features/steps/navigator_steps.py:30

  Then отображается маршрут "A" с расстоянием 0 km # features/steps/navigator_steps.py:35 0.000s
  Then отображается маршрут "A" с расстоянием 0 km # features/steps/navigator_steps.py:35
  Then система выдает ошибку "No route exists between start and end" # features/steps/navigator_steps.py:40 0.000s
  Then система выдает ошибку "No route exists between start and end" # features/steps/navigator_steps.py:40
```

Рисунок 18 — один сценарий не прошел

На этапе green нормально находить ошибки или баги в работе. Не прошедший сценарий сигнализирует о разнице вывода с ожидаемым результатом.

## Шаг 4. Рефакторинг

Уберём выскочивший баг, убрав .0 если километры — целые числа(рис.19)

```
def display_route(path: List[str], distance: float) -> str: 2 usages
    if not path:
        raise ValueError("Invalid path")
    route_str = " -> ".join(path)
    # Убираем .0 у целых чисел
    dist_str = str(int(distance)) if distance.is_integer() else str(distance)
    return f"Route: {route_str}, Distance: {dist_str} km"
```

Рисунок 19 — исправлен баг

Проверяем вывод(рис 20)

```
Feature: Поиск и отображение маршрута в навигаторе # features/scenario.feature

  Then отображается оптимальный маршрут "A -> B -> C" с расстоянием 8 km
  Then отображается оптимальный маршрут "A -> B -> C" с расстоянием 8 km
  Then отображается маршрут "A" с расстоянием 0 km # features/steps/
  Then отображается маршрут "A" с расстоянием 0 km # features/steps/
  Then система выдает ошибку "No route exists between start and end" # fe
  Then система выдает ошибку "No route exists between start and end" # fe

1 feature passed, 0 failed, 0 skipped
3 scenarios passed, 0 failed, 0 skipped
9 steps passed, 0 failed, 0 skipped
Took 0min 0.003s
```

Рисунок 20 — правильный вывод

## Заключение по BDD

BDD позволил в короткий срок автоматизировать выводы и проверки результатов, сразу выявляя несоответствия с ожидаемым результатом, сделав разработку приятней и удобней, а тестирование проще и комфортнее.



## 5. ЗАДАЧА ДЛЯ SDD

**Цель:** Проверить корректность `build_route` на множестве примеров, включая граничные случаи, с использованием генерации данных.

### Шаг 1. Сбор конкретных примеров и реализация тестов

Библиотека `hypothesis` позволяет обучить простенькую нейросеть на примерах для подстановки входных данных в алгоритм с автоматизированной проверкой. Нейросеть подставляет более 100 результатов и если каждый соответствует ожиданиям, то тест пройден. Составим 4 примера для разных случаев, чтобы научить нейронку (Таблица 1).

Таблица 1 — примеры для `hypothesis`

| Входные данные (граф, start, end)                              | Ожидаемый результат               |
|----------------------------------------------------------------|-----------------------------------|
| <code>{'A': {'B': 10}}, 'A', 'B'</code>                        | <code>(['A', 'B'], 10)</code>     |
| <code>{'A': {'B': 5, 'C': 15}, 'B': {'C': 3}}, 'A', 'C'</code> | <code>(['A', 'B', 'C'], 8)</code> |
| <code>{'A': {}}, 'A', 'A'</code>                               | <code>(['A'], 0)</code>           |
| <code>{'A': {'B': 5}}, 'A', 'C'</code>                         | <code>ValueError</code>           |

Теперь можно перейти к написанию тестов. В процессе разработки тестов я столкнулся с множеством проблем, связанных с нормализацией весов на узлах, поэтому пришлось разрешить большое количество несостыковок выходного ответа с ожидаемым и раз за разом тестировать новые варианты программы-теста, но конечный вариант выглядит так (рис.21-23):

```
# tests/test_navigator_sdd.py
from hypothesis import given, strategies as st, example
from navigator import build_route
import networkx as nx
import pytest

def generate_graph():
    return st.dictionaries(
        keys=st.text(min_size=1, max_size=1),
        values=st.dictionaries(
            keys=st.text(min_size=1, max_size=1),
            values=st.floats(min_value=0.1, max_value=1000),
            min_size=0, max_size=5
        ),
        min_size=1, max_size=10
    )

def normalize_graph(graph_dict):
    graph = {}
    all_nodes = set()

    for u, edges in graph_dict.items():
```

```

    if u not in graph:
        graph[u] = {}
    all_nodes.add(u)
    for v, w in edges.items():
        all_nodes.add(v)
        if v not in graph:
            graph[v] = {}
        min_w = min(w, graph[v].get(u, float('inf')))
        graph[u][v] = min_w
        graph[v][u] = min_w

    for node in all_nodes:
        if node not in graph:
            graph[node] = {}

    return graph

@given(graph=generate_graph())
@example(graph={'A': {}})
@example(graph={'A': {'B': 5}, 'B': {}})
@example(graph={'A': {'B': 10}, 'B': {'A': 15}})
def test_build_route_property(graph):
    all_nodes = set(graph.keys())
    for edges in graph.values():
        all_nodes.update(edges.keys())
    nodes = list(all_nodes)
    if not nodes:
        return

    start = nodes[0]
    end = nodes[-1] if len(nodes) > 1 else nodes[0]

    if start == end:
        path, dist = build_route(graph, start, end)
        assert path == [start] and dist == 0
        return

    normalized_graph = normalize_graph(graph)

    G = nx.Graph()
    for u, edges in normalized_graph.items():
        for v, w in edges.items():
            G.add_edge(u, v, weight=w)

    try:
        expected_dist = nx.shortest_path_length(G, start, end, weight='weight')
        path, dist = build_route(graph, start, end)

        assert abs(dist - expected_dist) < 1e-6

        assert path[0] == start
        assert path[-1] == end

        calculated_dist = 0.0

```

```

for i in range(len(path) - 1):
    u, v = path[i], path[i + 1]
    assert u in normalized_graph
    assert v in normalized_graph[u]
    calculated_dist += normalized_graph[u][v]

assert abs(calculated_dist - dist) < 1e-6

except (nx.NetworkXNoPath, nx.NodeNotFound):
    with pytest.raises(ValueError):
        build_route(graph, start, end)

```

'''

```

2   from hypothesis import given, strategies as st, example
3   from navigator import build_route
4   import networkx as nx
5   import pytest
6
7
8   def generate_graph(): 1 usage
9       return st.dictionaries(
10           keys=st.text(min_size=1, max_size=1),
11           values=st.dictionaries(
12               keys=st.text(min_size=1, max_size=1),
13               values=st.floats(min_value=0.1, max_value=1000),
14               min_size=0, max_size=5
15           ),
16           min_size=1, max_size=10
17       )
18
19
20  def normalize_graph(graph_dict): 1 usage
21      graph = {}
22      all_nodes = set()
23
24      for u, edges in graph_dict.items():
25          if u not in graph:
26              graph[u] = {}
27          all_nodes.add(u)
28      for v, w in edges.items():
29          all_nodes.add(v)
30          if v not in graph:
31              graph[v] = {}
32          min_w = min(w, graph[v].get(u, float('inf')))
33          graph[u][v] = min_w
34          graph[v][u] = min_w
35
36      for node in all_nodes:
37          if node not in graph:
38              graph[node] = {}
39
40      return graph
41

```

```

43 @given(graph=generate_graph())
44 @example(graph={'A': {}})
45 @example(graph={'A': {'B': 5}, 'B': {}})
46 @example(graph={'A': {'B': 10}, 'B': {'A': 15}})
47 ▶ def test_build_route_property(graph):
48     all_nodes = set(graph.keys())
49     for edges in graph.values():
50         all_nodes.update(edges.keys())
51     nodes = list(all_nodes)
52     if not nodes:
53         return
54
55     start = nodes[0]
56     end = nodes[-1] if len(nodes) > 1 else nodes[0]
57
58     if start == end:
59         path, dist = build_route(graph, start, end)
60         assert path == [start] and dist == 0
61         return
62
63     normalized_graph = normalize_graph(graph)
64
65     G = nx.Graph()
66     for u, edges in normalized_graph.items():
67         for v, w in edges.items():
68             G.add_edge(u, v, weight=w)
69
70     try:
71         expected_dist = nx.shortest_path_length(G, start, end, weight='weight')
72         path, dist = build_route(graph, start, end)
73
74         assert abs(dist - expected_dist) < 1e-6
75
76         assert path[0] == start
77         assert path[-1] == end

```

Рисунок 22

```

79     calculated_dist = 0.0
80     ▼ for i in range(len(path) - 1):
81         u, v = path[i], path[i + 1]
82         assert u in normalized_graph
83         assert v in normalized_graph[u]
84         calculated_dist += normalized_graph[u][v]
85
86         assert abs(calculated_dist - dist) < 1e-6
87
88     ▼ except (nx.NetworkXNoPath, nx.NodeNotFound):
89     ▼     with pytest.raises(ValueError):
90         build_route(graph, start, end)

```

Рисунок 23 — тесты SDD

С таким тестом, учитывая все возможные случаи и несостыковки, получим полную проходимость тестов (рис.24)

```
(.venv) PS C:\Users\aleks\PycharmProjects\PythonProject> pytest tests/test_navigator_sdd.py
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-9.0.0, pluggy-1.6.0
rootdir: C:\Users\aleks\PycharmProjects\PythonProject
plugins: hypothesis-6.147.0
collected 1 item

tests\test_navigator_sdd.py .
===== 1 passed in 0.80s =====
```

Рисунок 24 — результаты тестов

## Выводы по SDD

Лично для меня этот метод оказался самым тяжелым и долгим по реализации, но только он позволил доработать программу так, чтобы в любых граничных случаях получать верные ответы, так что это определенно самый действенный метод для продакшена, особенно если у исполнителя есть ресурсы на написание таких тестов для каждой новой фичи.

## **6.ОБЩИЕ ВЫВОДЫ ПО МЕТОДАМ**

TDD оказался самым быстрым и легким, позволил быстро привести алгоритм в рабочее состояние за минимальное время

ATDD сохраняет преимущества TDD, но дополнительно позволяет легче оценивать ситуацию для других участников разработки, в том числе заказчика.

BDD самый красивый, тесты выводятся на человеческом языке , что упрощает анализ

SDD самый ресурсоёмкий, но именно он нужен для продакшена, так как позволяет найти скрытые баги и несоответствия.